ARCHITECTURE REFERENCE · LAB-SHELL · NATS-TECH-LAB

The Application Shell & its Plugin Architecture

One Vue host, no feature imports. Everything a user sees beyond the frame arrives as a curated, signed, independently built micro-frontend plugin. These twelve cards state how that works, as built.

THE GOVERNING INVARIANT

Metadata before code. The shell must know a plugin's identity, contract compatibility, permissions, routes, navigation and extension assignments *before* it loads a single byte of that plugin's executable remote module.

AND WHAT IT IS NOT

This is a **trusted first-party plugin model, not a browser sandbox.** Runtime discovery removes host rebuilds; it does not make arbitrary remote JavaScript safe. Every card that touches trust says so again.

THE TWELVE CARDS

DRAWN FROM ARCHITECTURE-APP-SHELL.MD · BUSINESS_RULES-APP-SHELL.MD

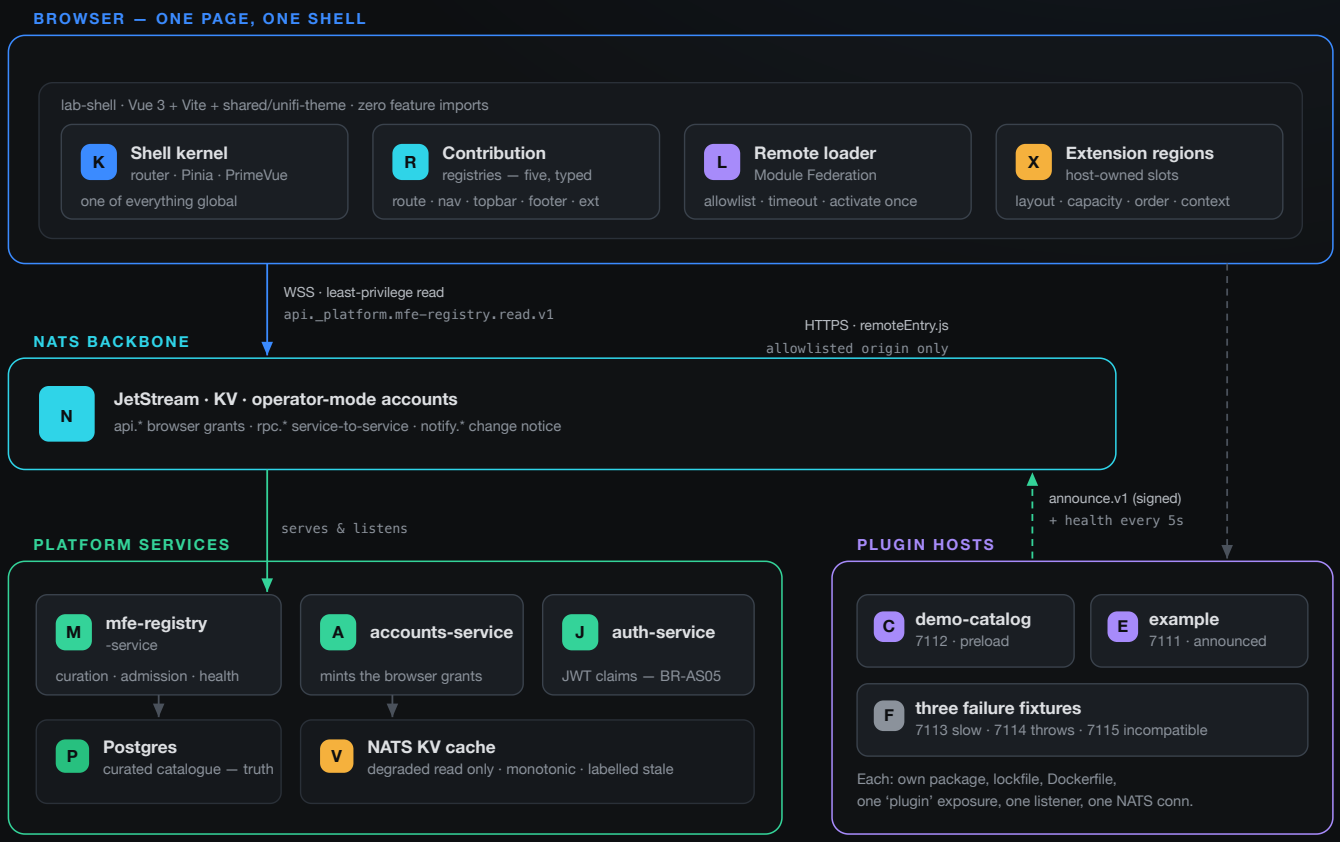
AS BUILT THROUGH PHASE 15 · 2026-09-02

01

THE SERVICES INVOLVED

What is actually running

Three platform services, one browser page, and one container per plugin. Nothing here talks to a plugin over HTTP except the browser fetching its code.



Solid = request/reply or a read the caller waits on · Dashed = pushed, fire-and-forget · Blue = browser plane · Cyan = backbone · Green = platform · Purple = plugin-owned
Boundary labels name the plane. Platform services share one backend Docker network; each plugin is its own container.

WHAT THE SHELL NEVER DOES

- Import a plugin at build time
- Open a business NATS connection for a plugin
- Accept a remote URL from any channel but the registry

WHAT THE REGISTRY NEVER DOES

- Execute plugin code
- Mint or hold credentials
- Enable an entry on a publisher's say-so

WHAT A PLUGIN NEVER DOES

- Dial the browser — the browser dials it
- Set its own `enabled`, `lifecycle` or `revision`
- Reach another plugin's connection

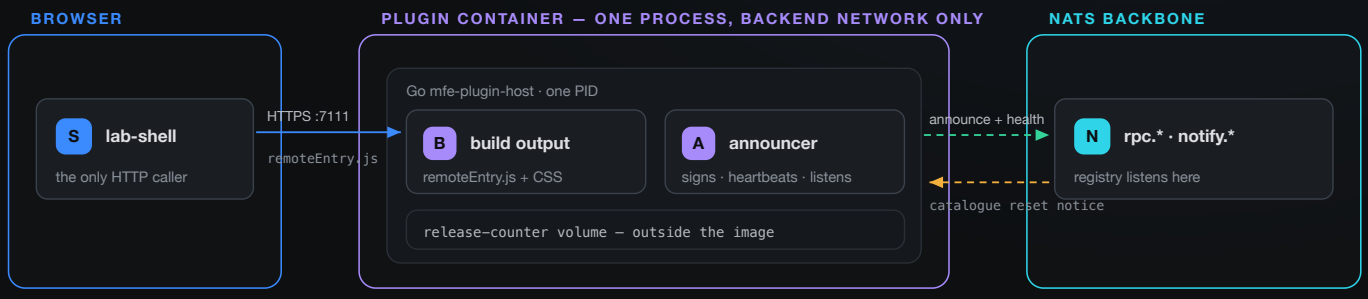
Every arrow crossing a boundary is either an allowlisted asset fetch or a NATS subject. There is no third way in, and that is the whole security perimeter.

02

ONE CONTAINER, TWO LINKS

What a plugin is made of

A plugin is one process holding exactly two links: an HTTP listener the browser dials, and one NATS connection it dials out on. Phase 14 merged the containers; the counts did not change.



Solid = the browser waits on it · Dashed = pushed, fire-and-forget · Counts per plugin: 1 HTTP listener, 1 NATS connection — unchanged by the Phase 14 merge

The container joins the backend Docker network only. Since Phase 15 the registry makes no outbound call to it at all.

MANIFEST.JSON — THE PLUGIN OWNS THIS

- Identity, version, remote entry, contributions
- Served at `/manifest.json` from the plugin itself
- **Cannot claim** `source`, `lifecycle`, `enabled` or `revision`
- The release number is injected at runtime, just before signing — not baked into the image

REGISTRY.JSON — THE OPERATOR OWNS THIS

- A preload wrapper holding manifests plus enablement
- Since Phase 13 it wraps `demo-catalog` alone
- Editing it and restarting **changes no existing row**
- It is also the single input to the explicit operator seeder

WHY THE MERGE WAS WORTH DOING

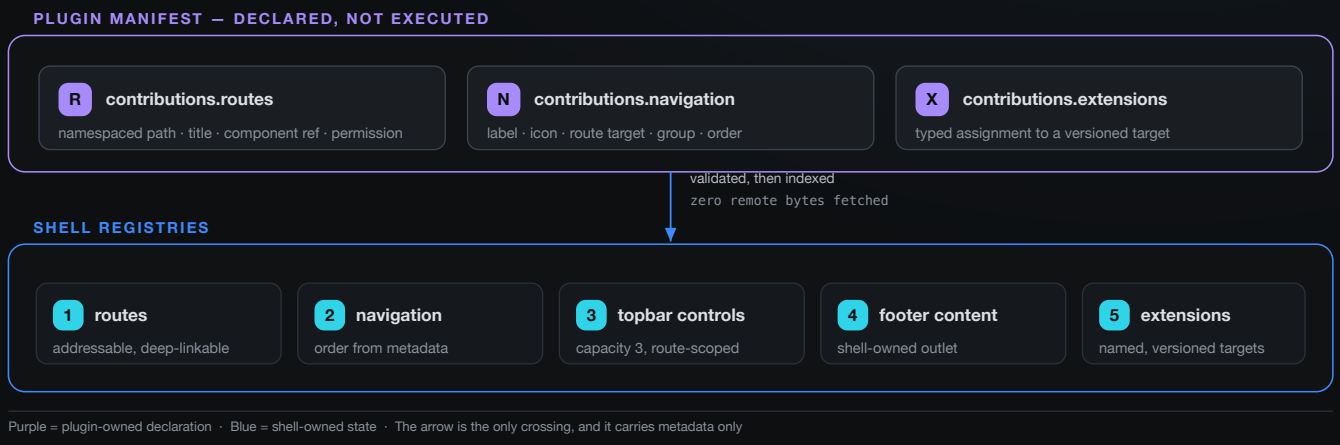
- Before Phase 14, five announced plugins meant **ten containers** — an nginx frontend and an announcer sidecar that shared nothing but a plugin id.
- The two links were always two different protocols in two different directions. Merging changed **who holds them, not how many there are**.
- Phase 15 then deleted the last reason a plugin joined the frontend network: the registry's `GET /healthz` probe and its hand-written origin map.

03

THE INTEGRATION SURFACE

How UI is declared

A plugin extends the application through one published contract and nothing else. It declares contributions as metadata; the shell sorts them into five separately typed registries.



Purple = plugin-owned declaration · Blue = shell-owned state · The arrow is the only crossing, and it carries metadata only

DECISION

Five typed registries, not one global bag. Each contribution kind is explicit and independently validated, so a bad entry of one kind cannot poison another. Rejection is per contribution, never per plugin and never per registry.

IDENTITY & ORDER

- Every plugin and contribution ID is **namespaced**
- Pinia store IDs become `{plugin-id}/{store-id}` — three apps each had a store called `tenant`
- Render order comes from accepted metadata, **not** from remote load timing

WHAT IS REFUSED

- A duplicate plugin ID — the whole entry
- A duplicate or malformed contribution — that one only
- A contribution aimed at an undeclared or wrong-version target
- Every refusal is recorded in `refusals`, never silently dropped

NOT A SANDBOX — SAY IT PLAINLY

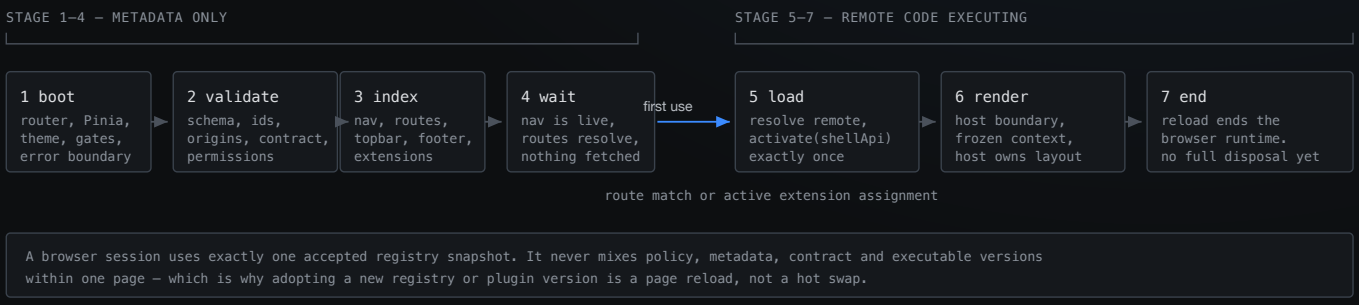
- Plugin code runs in the shell's own JS realm
- It can reach `document`, so it **could** mount a second shell or write outside its container
- Nothing in the repo prevents that. The rule is a contract, held up by curation and review

04

BOOT TO RENDER

When the code actually loads

Seven stages. The first four fetch no remote code at all — navigation and routes are already live and clickable before a single plugin byte exists in the page.



A browser session uses exactly one accepted registry snapshot. It never mixes policy, metadata, contract and executable versions within one page — which is why adopting a new registry or plugin version is a page reload, not a hot swap.

THE COLD DEEP LINK

- A URL into a not-yet-loaded remote resolves from **metadata**, then loads
- The route is addressable before the code exists — that is the point of BR-AS12
- Pending regions animate rather than showing an empty frame

THE SHELL API A PLUGIN RECEIVES

- One shared frozen object: `{ version: 1, ui: { ExtensionRegion } }`
- No connection, no credential, no host registry
- Deliberately narrow — and still not a sandbox for same-realm code

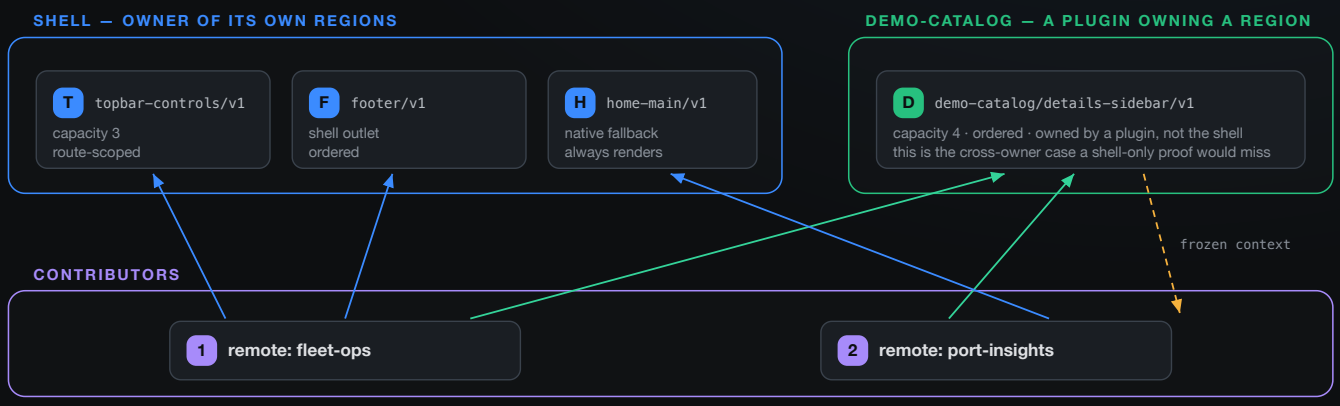
Lazy is not an optimisation here, it is the trust model. Declaring a route or a nav entry downloads nothing, so a refused entry costs zero executed bytes.

05

HOST-OWNED REGIONS

The host decides where

Plugins fill targets. They never choose where a target lives. A region's owner declares it, versions it, sets order and capacity, and freezes the context it hands out.



Solid = a placed contribution · Dashed amber = the frozen readonly context · Blue = shell-owned target · Green = plugin-owned target

Contributors are independent remotes with no knowledge of each other. One of them fills a shell target and a catalog target under the same contract.

WHAT AN OWNER DECLARES

- A stable, versioned target ID — `{owner}/{region}/v{major}`
- The accepted contribution type
- Layout, capacity and deterministic order
- The readonly context, and any permission constraint

WHAT A CONTRIBUTOR CANNOT DO

- Query for a DOM element and attach itself
- Choose its own position or reorder siblings
- Mutate the context — it is frozen on the way in
- Exceed capacity quietly: overflow is **reported**

WHEN AN OWNER IS WITHDRAWN

- Only placements **into that slot** suspend
- Their contributors stay active everywhere else
- Eligible placements return **exactly once** when the unchanged slot returns

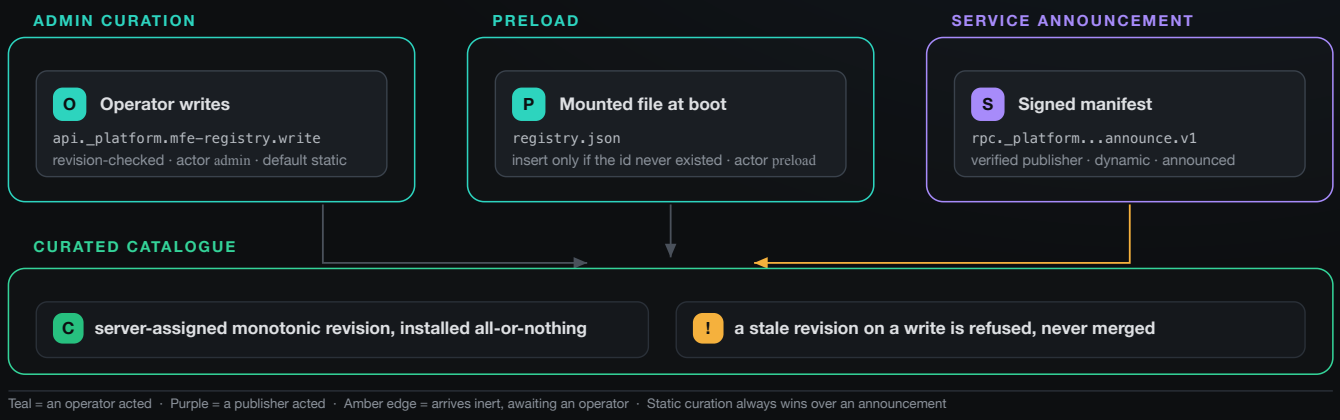
Host ownership is what preserves accessibility, visual fidelity and local layout control. It is also what lets one plugin safely host another plugin's UI.

06

THREE TIERS, ONE CATALOGUE

How an entry gets in

Three paths write the same curated registry and all three pass the origin allowlist. They differ in authority and in what they default to — not in where they land.



Teal = an operator acted · Purple = a publisher acted · Amber edge = arrives inert, awaiting an operator · Static curation always wins over an announcement

DECISION

Announcement is not activation. A publisher can report its own manifest. It cannot grant itself permission to execute. An enabled dynamic entry may update within its approved origin; an origin change sends it back to `announced` until an operator reapproves.

A FRESH LAB, FIRST BOOT

- `demo-catalog` is the **only** plugin served
- Every announced candidate is stored **disabled** and stays that way
- Once enabled, Postgres keeps that decision across restarts
- The shell says so on screen — otherwise a correct first run reads as broken

THE SEEDER, AND WHY IT WRITES NOTHING TWICE

- Reads first, applies only the operations actually missing
- A `revoked` key is never handed its trust back
- A transferred plugin stays with its new owner
- Re-running against a converged registry performs **zero** writes

07

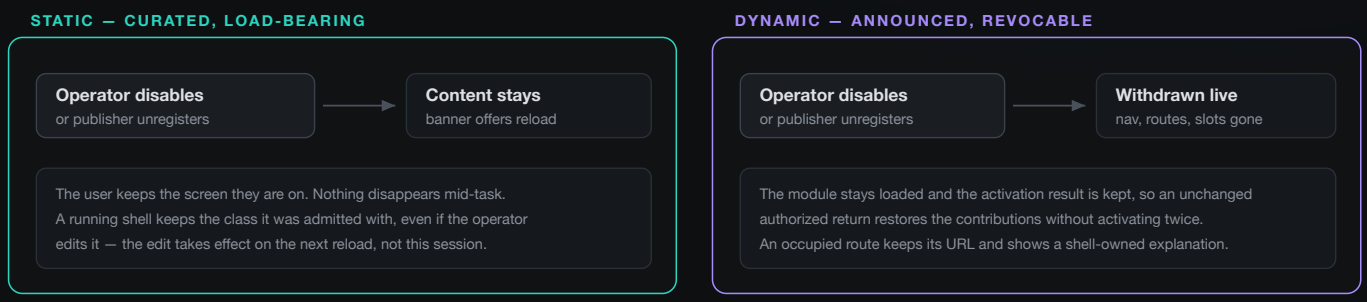
LIFECYCLE CLASS

Static and dynamic

Lifecycle is explicit platform metadata, set by the operator and independent of how the plugin arrived. It answers exactly one question: may this plugin's UI vanish under the user's hands?

DECISION

A static plugin is never unloaded live — it is offered a reload. A dynamic plugin's contributions are withdrawn on the spot. One exception, and only one: a security revocation forces the reload on both (BR-AS49).



EVENT → BEHAVIOUR, BY CLASS

EVENT	STATIC	DYNAMIC
Operator disable	Keep content, offer reload	Withdraw contributions live
Signed publisher unregister	No live withdrawal — curation is authoritative	Withdraw live, keep operator approval
Service stop / failed health / lost NATS	Decorate only	Decorate only
Unchanged authorized return	Retract the obsolete reload offer	Restore cached contributions, no second activate
Changed runtime definition	Offer reload	Offer reload — never hot replacement
Operator class edit	Offer reload; this session retains its admitted class	
Publisher / key revocation	Forced reload on both (BR-AS49) — the banner takes it, it does not offer it	

WHERE STATIC COMES FROM

- **Preload** at service boot — insert only if the id never existed
- **Admin curation** — an operator typed it
- Legacy unclassified entries default here

WHERE DYNAMIC COMES FROM

- **Announcement** — a signed manifest over `rpc.*`
- Enters `announced`, withheld until enabled
- Origin change sends it back to `announced`

WHAT WITHDRAWAL IS NOT

- No `scope.dispose()`, no JS isolation
- An in-flight callback is not interrupted
- The only promise: **gone at the next paint**

CARD 07

Static protects the user's work. Dynamic protects the platform's control. The operator picks which matters more for each plugin, and the choice is metadata — not a rebuild.

08

SEVEN OBSERVABLE STATES

What the shell can say about a plugin

Every plugin the shell knows about is in exactly one of seven states, and each one is visible to an operator. Two of them look the same to a user and must never be merged.

WHY INCOMPATIBLE ≠ FAILED

- **incompatible** — the shell refused it. Terminal for the session. **No remote code ran.**
- **failed** — the shell tried and it broke. Retryable.
- Collapsing them would tell an operator to go debug a plugin the shell never executed.

AVAILABLE MEANS ZERO BYTES FETCHED

- Navigation entries and routes are **already live** at `available`
- The remote is fetched on first use, not at boot
- A disabled plugin costs a boot nothing

WHAT WITHDRAWAL REMOVES, AND WHAT IT KEEPS

REMOVED	KEPT
Navigation entries the plugin owns	The loaded module — a re-entry does not re-import
Routes, for new navigation	The activation result — no second `activate()`
Extension placements, controls, footer items	A `withdrawn` record, so the reason is auditable

A late-completing import or activation cannot register contributions for a withdrawn plugin. Duplicate events are idempotent.

An **occupied route** keeps its URL and shows a shell-owned explanation with a link back. Component and form state is **not** promised to survive.

The state a plugin is in is a platform fact, not a guess from symptoms. That is what makes a failure report actionable instead of a support conversation.

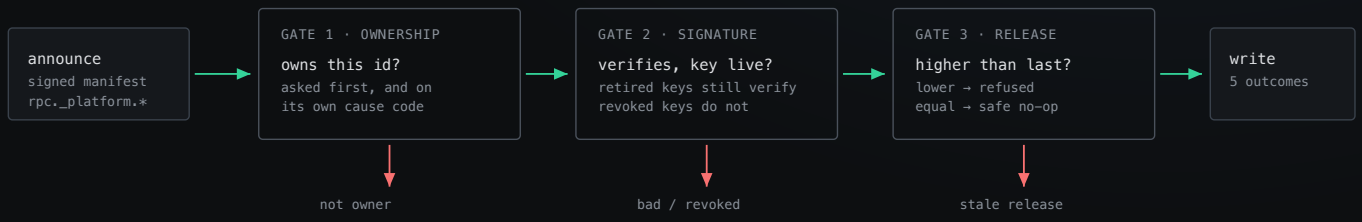
09

THREE TRUST GATES

Who published this manifest

An announced manifest passes three gates, in order, before it touches the catalogue. All three run in the service. None of them run in the browser.

Gates are re-checked where the write commits. Lock order is always publisher_revision, then registry.revision — the announce path takes only the second, so they cannot deadlock.



FIVE OUTCOMES, ALL AUDITABLE

- `inserted` — new, and awaiting an operator
- `pending` — stored, not yet enabled
- `updated` — an approved entry changed in place
- `requeued` — origin changed, back to `announced`
- `ignored` — nothing to do; retry is safe

WHAT SIGNING DOES NOT COVER

- Not what the code **does** — only who published the manifest
- Not the remote it points at: those bytes are unsigned and unchecked. The **origin allowlist** is what bounds where code comes from
- Deliberately not bound to the announcing service's identity
- Deliberately not the NATS trust chain — a leaked publisher key cannot connect to NATS as anything

DECISION

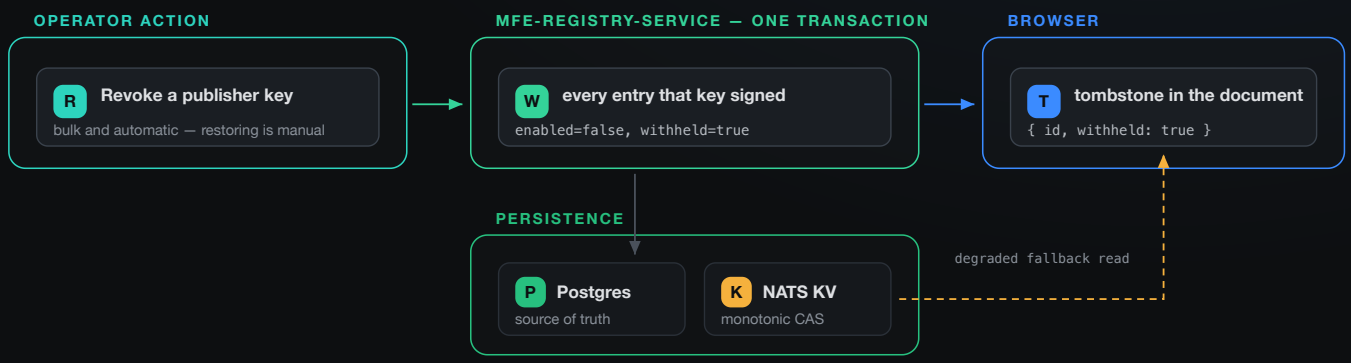
Ownership is answered before the signature. A refusal must not become a way to enumerate which plugin ids are registered — and an operator has to be able to tell a forged signature from a genuine signature over somebody else's plugin.

10

WITHDRAWAL & DEGRADED READ

Taking it back

Revoking a key withholds, in one transaction, every entry that key ever signed. The shell learns about it in the document itself — never only in a notification.



A tombstone travels in the document, not in the change notice — a notify is fire-and-forget, and a revocation must not depend on a message arriving.
When Postgres is unavailable the read falls back to the KV cache and the shell says "degraded, as of revision N". Refusing to serve would turn a rare security event into a routine outage.

WITHHELD ≠ DISABLED

- **disabled** — not reviewed yet
- **withheld** — we withdrew this
- Only the second unloads anything — but one revocation writes both, in one statement

TOMBSTONES OVERRIDE THE STATIC RULE

- The shell treats a tombstone for a running plugin as a **forced** reload
- That beats the static "never unload live" rule
- The only promise is that the plugin **stops at the next paint** — not runtime isolation

WHY DEGRADED READING IS SAFE ENOUGH

- Cache writes are monotonic — a lower revision is refused
- Staleness is **visible**, never silent
- A degraded document may say what was **withdrawn**, never what **exists**; it can raise a forced reload and never retracts one

Refusing to serve at all would turn a rare security event into a routine outage. Admin has no degraded label at all — its curated read goes straight to Postgres.

11

COMPATIBILITY & RESOLUTION

What a running shell may change about itself

Compatibility is decided before any remote code runs, per entry, never per registry. And a live shell applies almost nothing to itself — it offers a reload instead.

DECISION

A new entry id is the only difference a running shell may apply to itself. Label, order, route prefix, permission, version, remote, contribution list — every one of those is a reload offer. It is written as a whitelist on purpose, because the write path is `ON CONFLICT DO UPDATE` and a blacklist would silently miss a new column.

A revision change is published on notify._platform.mfe-registry.frontend-plugins.changed and read conditionally. Every reconnect reads unconditionally. No focus watcher, no interval poll, no HTTP fallback.

FAILURE MODEL — WHAT HAPPENS, AND HOW FAR IT SPREADS

FAILURE	CONTAINMENT
Registry unavailable	Native Home and Plugins still render, with a diagnostic
One invalid entry	Reject that entry, continue with its siblings
Duplicate id	Reject the conflicting entry
Remote load timeout	Local, retryable fallback — the rest of the shell is unaffected
activate() throws	Roll back the partial registration
A contribution fails to render	Contained at the route or extension-host boundary

RELEASE COUNTERS GO ONE WAY

- A publisher owns its counter and it only goes **up**
- A withdraw-then-return spends **three** numbers: announce, unregister, re-announce
- The release is injected at runtime, just before signing

A RESYNC THAT CHANGES NOTHING

- Result is `converged` : no revision, no audit row, no notification, no cache refresh
- **The accepted release still advances** — it is the watermark the backwards and reused checks read
- `converged` and `NoOp` stay distinct on the wire and must not be collapsed

CARD 11

Offers are synced with each successful read, not accumulated. A read that carries no document — `unchanged` , or degraded — retracts nothing. The policy is a pure function: `shell/registry/readPolicy.js` .

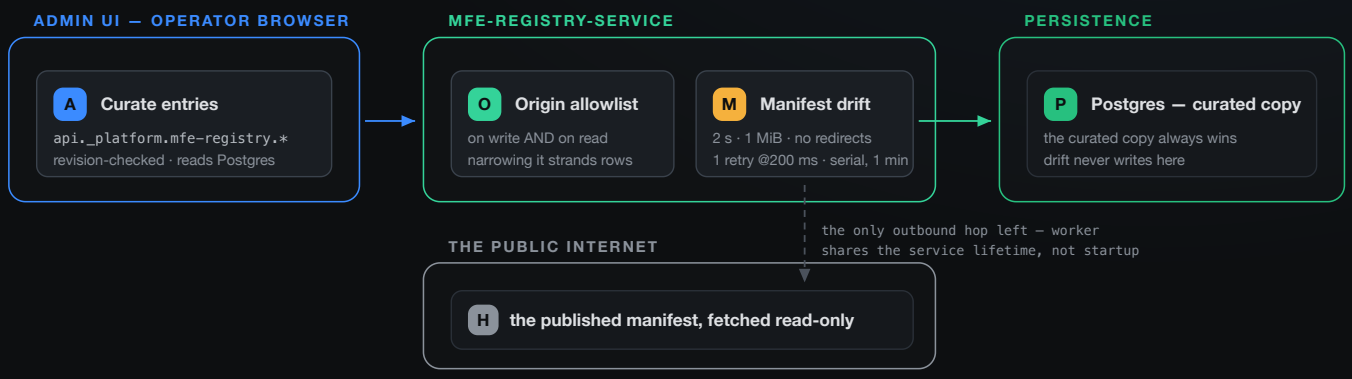
BR-AS13 · AS17–19 · AS67

12

THE OPERATOR SURFACE

Curation, drift, and the one HTTP call left

The registry is service state, not a build artifact. An operator adds or removes a curated entry and a newly booting shell sees it — without restarting any service.



REGISTRY_FETCH_ORIGINS maps an already-allowlisted browser origin to a service-reachable one. It cannot grant an origin, and a missing mapping never falls back to browser localhost.

THE MANIFEST COLUMN SAYS ONE OF THREE THINGS

- `checked` — the published manifest agrees with the curated copy
- `drift` — it disagrees, naming the differing fields
- `not checked` — no observation yet
- Observations are in memory, invalidated by a curated edit, exposed only through `EntryView`

DRIFT CHANGES NOTHING ON ITS OWN

- Refresh reads that snapshot — it never initiates HTTP itself
- Neither disagreement nor a failed fetch writes to the catalogue
- Neither changes whether a plugin is served
- It is an **observation for an operator**, not an enforcement point

CARD 12

The boot-time seeder writes through the same curated path under the shared operator identity — not a fourth tier, and not a boot-time bypass of the revision check or the audit trail. It converges, and it never reverses an operator's decision.

BR-AS16 · AS20 · AS45 · AS68